

```
$ onboard astrid-runtime/astrid --profile dev
```

Astrid : un runtime Rust capability-secure pour capsules WebAssembly, porté par un seul mainteneur

Astrid is a portable, capability-secure operating system for composable software.

10 277

ÉTOILES GITHUB

3

CONTRIBUTEURS ACTIFS

sur 12 semaines

24

PR OUVERTES

9 sept. 2026

DERNIER COMMIT

Que fait ce projet et pour qui ?

> Quatre binaires Rust (`astrid` , `astrid-daemon` , `astrid-build` , `astrid-emit`) autour d'un seul principe, une capsule WebAssembly n'a que les capacités qu'on lui accorde ; 10277 étoiles et 216 issues ouvertes pour un dépôt créé le 2026-02-15.

Astrid se décrit comme « a portable, capability-secure operating system for composable software » : aujourd'hui un runtime user-space sur macOS et Linux, la cible affichée étant un OS autonome.

Les composants sont des **capsules** WebAssembly exécutées dans Wasmtime sans autorité ambiante ; le noyau (`astrid-daemon`) se contente de router des événements IPC sous ACL de capacités, sans handle LLM ni registre d'outils.

Public visé : qui construit des systèmes d'agents, des outils, des services ou une distribution, et refuse que le modèle soit l'autorité qui autorise ses propres actions ; Astrid n'impose ni produit, ni fournisseur de modèle, ni boucle d'agent. Topics cohérents : `rust` , `wasm` , `mcp` , `capability-security` .

Quatre binaires : `astrid` (CLI uplink, TUI et mode headless), `astrid-daemon` (noyau), `astrid-build` (compile les capsules vers `wasm32-unknown-unknown`), `astrid-emit` (pont stdio → bus).

Chiffres : 10277 étoiles, 137 forks, 216 issues ouvertes, dépôt créé le 2026-02-15 par une organisation, dernier push le 2026-09-09.

Licence relevée par GitHub : Apache-2.0 ; le README et les fichiers `LICENSE-APACHE` / `LICENSE-MIT` disent « MIT OR Apache-2.0 ». Donnée non relevée : la répartition des langages ; à vérifier sur <https://github.com/astrid-runtime/astrid>. Le manifeste est un workspace Cargo.

Comment le code est-il organisé ?

> 33 crates sous `crates/`, 32 dans le workspace ; les crates de sécurité (`astrid-crypto`, `astrid-capabilities`, `astrid-core` ...) sont fermées aux nouveaux, votre terrain est `crates/astrid-cli`.

- └ `.cargo/` config Cargo locale
- └ `.gemini/` config agent Gemini
- └ `.github/` 20 workflows GitHub Actions
- └ `changes/` notes de changement en attente
- └ `container/` définitions de conteneur
- └ `crates/` code principal : 33 crates Rust, 32 membres du workspace
- └ `docs/` documentation
- └ `e2e/` scénarios TOML et fixtures bout en bout
- └ `fuzz/` tests de fuzzing
- └ `native/` code natif macOS
- └ `release/` config de release (nightly.toml)
- └ `scripts/` scripts build, CI, release
- └ `.dockerignore` exclusions du build Docker
- └ `.gitattributes` attributs git
- └ `.gitignore` exclusions git
- └ `.gitmodules` sous-modules git

... et 12 autres entrées à la racine du dépôt

Workspace Cargo (`Cargo.toml`, resolver 2, 32 membres) ; tout le code est sous `crates/` : 33 crates, dont `astrid-cli-mockup` hors workspace.

Repères par rôle, d'après les noms de crates : noyau et bus (`astrid-kernel`, `astrid-daemon`, `astrid-runtime`, `astrid-events`, `astrid-hooks`, `astrid-config`) ; sécurité (`astrid-crypto`, `astrid-capabilities`, `astrid-audit`, `astrid-approval`, `astrid-vfs`, `astrid-core`) ; état (`astrid-storage`, `astrid-storage-provider-fuse` / `-fskit` / `-winfsp`, `astrid-storage-chunker-evidence`) ; frontends (`astrid-cli`, `astrid-gateway`, `astrid-mcp`, `astrid-uplink`) ; outillage capsules (`astrid-build`, `astrid-capsule`, `astrid-capsule-install`, `astrid-capsule-types`, `astrid-emit`).

Frontière de sécurité, réservée aux tiers Core et Maintenir : `astrid-crypto`, `astrid-capabilities`, `astrid-audit`, `astrid-approval`, `astrid-vfs`, `astrid-storage`, `astrid-sys`, `astrid-core`. Note : `astrid-sys` n'apparaît pas dans le `crates/` relevé.

Autour du code : `e2e/` (scénarios TOML et fixtures bout en bout), `fuzz/`, `crates/astrid-integration-tests` (39 fichiers, cargo test + insta), `native/macos/`, `docs/`, `scripts/` (build, CI, release), `changes/` (fragments de changelog en attente), `.github/` avec 20 workflows.

Deux curiosités : `wit` est un fichier de 0 octet à la racine et `.gitmodules` déclare des sous-modules ; leur cible n'est pas relevée, à vérifier dans <https://github.com/astrid-runtime/astrid/blob/main/.gitmodules>.

Aucun repère pour agents IA (AGENTS.md, CLAUDE.md, .cursorrules...) : score 0 sur 7 ; seul un dossier `.gemini/` existe.

Par où commencer à lire ?

> Deux fichiers suffisent pour le premier jour : `crates/astrid-cli/src/main.rs` (le client) puis `crates/astrid-daemon/src/main.rs` (26 lignes, la porte du noyau).

1. `crates/astrid-cli/src/main.rs` : binaire `astrid`, `#[tokio::main]` qui parse clap puis route vers `dispatch` ; client léger qui se connecte au kernel (auto-démarrage), crée ou reprend des sessions et rend les événements en streaming.
2. `crates/astrid-cli/src/cli.rs` : définitions clap des sous-commandes (`agent` , `capsule` , `quota` , `status` , `start` , `stop` , `ps` , `top`) ; déduit du `mod cli` de `main.rs`, fichier non lu.
3. `crates/astrid-daemon/src/main.rs` : 26 lignes, `setuid` puis runtime tokio multi-thread qui appelle `astrid_daemon::run()` . C'est la porte vers le noyau.
4. `crates/astrid-cli-mockup/src/main.rs` : maquette de CLI hors workspace, à ignorer au début.

Pour le modèle mental, lire d'abord les sections « How it works » et « The security model » du README (uplinks, bus, `Capsule.toml` avec `[imports]` / `[exports]` , ABI WIT `astrid:*`), puis The Astrid Book, présenté comme la référence canonique avec ancrs fichier/ligne¹. Le SDK des capsules (`astrid-sdk`) vit dans un autre dépôt².

Données non relevées : le corps de `astrid_daemon::run()` et le contenu de `crates/astrid-kernel/src/lib.rs` ; à vérifier dans <https://github.com/astrid-runtime/astrid/tree/main/crates/astrid-daemon/src> et <https://github.com/astrid-runtime/astrid/tree/main/crates/astrid-kernel/src>.

Comment builder, tester et lancer ?

```
> cargo build --release , cargo test --workspace -- --quiet , puis cargo clippy --workspace --all-features --all-targets --locked -- -D warnings : exactement ce que la CI rejoue sur chaque push et pull_request.
```

Rust 1.95 minimum, édition 2024, version épinglée par `rust-toolchain.toml` à la racine. Depuis un shell :

```
git clone https://github.com/astrid-runtime/astrid
cd astrid && cargo build --release # binary at ./target/release/astrid
cargo test --workspace -- --quiet
astrid start && astrid status && astrid stop
```

Commandes prises dans,, et le « Quick start » du README. Alternatives : `brew tap astrid-runtime/tap && brew install astrid` , ou `cargo install astrid` .

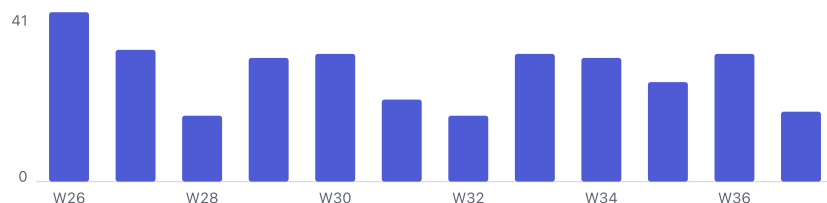
Avant une PR, rejouer la CI en local : `cargo fmt --all -- --check` , `cargo clippy --workspace --all-features --all-targets --locked -- -D warnings` , `cargo test --workspace --locked` ; CONTRIBUTING exige au minimum `cargo test --workspace` et `cargo clippy -- -D warnings` . Lints du workspace : `unsafe_code = "deny"` , `clippy pedantic` en warn, `arithmetic_side_effects = "deny"` .

CI : `.github/workflows/ci.yml` , déclenchée sur `push` (branches `main` et `stream-*`) et `pull_request` ; 14 jobs dont check, fmt, clippy, test (ubuntu et macos), msrv 1.95, cargo-audit, portabilité wasm32, tests natifs Windows. 20 workflows au total.

Données non relevées : les prérequis système (FUSE, FSKit, WinFsp) des tests de stockage et le temps de build ; à vérifier dans `scripts/` et <https://github.com/astrid-runtime/astrid/blob/main/github/workflows/ci.yml>.

Comment le projet vit-il ?

> 309 commits en 12 semaines pour `joshuajbouw`, 7 pour le suivant : bus factor 1, un seul relecteur possible, mais 188 PR fusionnées en 30 jours et un commit le 2026-09-09.



Activité soutenue : entre 16 et 41 commits par semaine sur les 12 dernières semaines (2026-W26 à 2026-W37), dernier commit le 2026-09-09T13:46:34Z.

Mais une seule personne porte le projet : `joshuajbouw` 309 commits, `jvsteiner` 7, Copilot 3. Bus factor 1, risque noté `high` : « Un seul contributeur porte la moitié des commits sur 12 semaines ».

Conséquence pratique : un seul relecteur pour vos PR ; 24 PR ouvertes pour 188 fusionnées en 30 jours. Donnée non relevée : combien attendent une revue ; à vérifier dans <https://github.com/astrid-runtime/astrid/pulls>.

Releases : `v2026.9.0` publiée le 2026-09-09, première du schéma `year.month.patch` annoncé par le README ; avant elle, de `v0.9.3` (2026-07-06) à `v0.10.4` (2026-07-20), plus des tags de canal `channel-stable`, `channel-nightly`, `channel-dev`.

Dépôt créé le 2026-02-15, 10277 étoiles pour 137 forks : beaucoup de regards, peu de mains. Données non relevées : l'historique avant W26 et un éventuel rythme de release annoncé ; à vérifier dans `CHANGELOG.md` (410 Ko) et <https://github.com/astrid-runtime/astrid/releases>.

Quelle première contribution ?

> Visez #459 (parser Markdown de la TUI, dans `crates/astrid-cli`) : commentez l'issue, attendez l'assignation, puis PR signée `git commit -s` avec `Closes #459` et le fragment `changes/459.fixed.md`³.

Processus strict : issue d'abord, attendre l'assignation par un mainteneur, label `newcomer-approved` posé sur la PR d'un nouveau, `Closes #N`, template PR entièrement rempli, commit signé `git commit -s` (DCO), fragment `changes/{issue}.{kind}.md`, titre Conventional Commits ; une PR sans issue liée est fermée.

5 issues `good first issue` sur 192 ouvertes (149 fermées en 30 jours, 42 étiquetées `bug`) :

- #459 « Markdown parser in TUI fails on numbered lists >= 10 »³ : la TUI est dans le binaire `astrid`, donc `crates/astrid-cli`, crate non-core ouverte aux nouveaux ; périmètre borné, test de régression évident : meilleur candidat.
- #276 « MCP interceptor: propagate tool_result.is_error instead of treating it as success »⁴ : probablement `crates/astrid-mcp`, hors liste des crates de sécurité.
- #497 « Remove dead AgentHandler stub from astrid-hooks »⁵ : `astrid-hooks` est un crate core, à confirmer avec le mainteneur avant de la réclamer. Restent #476 et #461.

Repère de style : les PR fusionnées suivent `fix(scope): ...` / `feat(scope): ...`, ex. « `fix(init): honor explicit grants after completed batch resume` » fusionnée le 2026-09-09. Éviter au début les fils chauds comme #1708 (29 commentaires).

Donnée non relevée : l'assignation éventuelle de ces issues ; à vérifier sur chaque URL avant de commenter pour la réclamer.

\$ Vos 3 premières actions

1. Cloner et vérifier le build : `git clone https://github.com/astrid-runtime/astrid` && `cd astrid` && `cargo build --release` , puis `cargo test --workspace -- --quiet` ; Rust 1.95 minimum, épinglé par `rust-toolchain.toml` .
2. Lancer le runtime et suivre le code qui tourne : `astrid start` && `astrid status` && `astrid capsule list` && `astrid stop` , en lisant `crates/astrid-cli/src/main.rs` puis `crates/astrid-daemon/src/main.rs` .
3. Réclamer #459 par un commentaire sur l'issue et attendre l'assignation ; ensuite seulement : `cargo clippy -- -D warnings` vert, `git commit -s -m "fix(scope): ..."` , fragment `changes/459.fixed.md` avec `Closes #459` , template de PR rempli, label `newcomer-approved` posé par un mainteneur³ .

Hors périmètre tant que vous n'êtes pas promu : `astrid-crypto` , `astrid-capabilities` , `astrid-audit` , `astrid-approval` , `astrid-vfs` , `astrid-storage` , `astrid-sys` , `astrid-core` , réservés aux tiers Core et Maintainer.

RÉFÉRENCES PUBLIQUES

Sources

1. <https://github.com/astrid-runtime/book>
2. <https://github.com/astrid-runtime/sdk-rust>
3. <https://github.com/astrid-runtime/astrid/issues/459>
4. <https://github.com/astrid-runtime/astrid/issues/276>
5. <https://github.com/astrid-runtime/astrid/issues/497>